Software Quality - Project

Documentation

Team : Balan Florin
Ciudin Stefana
Jitca Diana

Phase 1 - application development

This application is designed to assist users in **manually creating** and **managing the faculty timetable**. It offers a graphical interface for organizing and editing essential elements such as students (by year and group), teachers, disciplines, class types (course, seminar, laboratory), rooms, and time slots. The system enforces key scheduling rules—for instance, classes are only allowed Monday to Friday between 8:00 and 20:00, courses must be held in course rooms, and labs in laboratory rooms. It also provides real-time feedback by highlighting scheduling conflicts (e.g., room or teacher overlaps, broken restrictions) - more on these restrictions will be added in a later part of the documentation. The app focuses on functionality and clarity rather than automation, empowering the user to retain full control over the timetable creation process.

In this part of the documentation, we will focus on how we developed the application, its structure and the technologies used.

Application backend

This application is structured as a server-side system designed to manage and serve data for faculty-related operations. Its key components can be categorized into three main layers: **Core Initialization (Main Class)**, **API Layer**, and **Database Layer**.

1. Core Initialization (Main Class)

- `Main.java` serves as the application entry point.
- It initializes the **Database Layer**, ensuring the database schema and initial data setup is complete.
- It starts the **API Layer**, which sets up the REST API server using the **Javalin** framework.

2. API Layer

- The `RestApi.java` class creates and manages the REST API server using Javalin.
    - It handles Cross-Origin Resource Sharing (CORS) configuration and global exception handling.
    - Controllers (subclasses) are dynamically registered. Each controller serves routes for specific domain models (e.g., FacultyGroups, Rooms, etc.).
- The **Controllers** (e.g.,) interact with the **Database Layer** through manager classes to handle CRUD operations.
    - Each controller defines, for its resource:

        - Endpoints for fetching, adding, updating, and deleting data.
        - Methods that use context (`Context` from Javalin) to handle HTTP requests and responses.
    - All controllers inherit from an abstract base class **`AbstractController`**:
        - This base class defines a common structure and provides a `path` attribute to identify the API endpoint's root path, such as `/teachers` or `/rooms`.
        - Each controller implements the method to map specific HTTP routes (`GET`, `POST`, `PATCH`, `DELETE`) to handler functions. `registerEndpoints`
    - Handlers in controllers follow the pattern:
        - Extract parameters from the incoming HTTP request
        - Interact with the respective **manager classes** to perform database operations.
        - Return HTTP responses (e.g., JSON format for successful queries or appropriate status codes for errors).
    - Each controller manages a specific resource, and its endpoint paths are dynamically determined by the controller's `path` attribute (`/teachers`, `/schedules`).
    - Based on the success or failure of the request, responses use proper status codes:
        - `200 OK`: Success for retrieval operations.
        - `201 Created`: Successful creation of a resource.
        - `204 No Content`: Successful update or deletion with no content to return.
        - `404 Not Found`: Resource not found in the database.
        - `400 Bad Request`: Invalid input or query parameters.

3. Database Layer

- The **`Database.java`** class is a Singleton that manages the entire database connection and provides access to manager classes.
- It executes:
    - Database initialization and setup (e.g., creating tables, importing initial data).
    - Transaction handling during operations.

When creating the database structure, we tried to make it as simple and as clear as possible. The application uses **SQLite** as its database management system. The

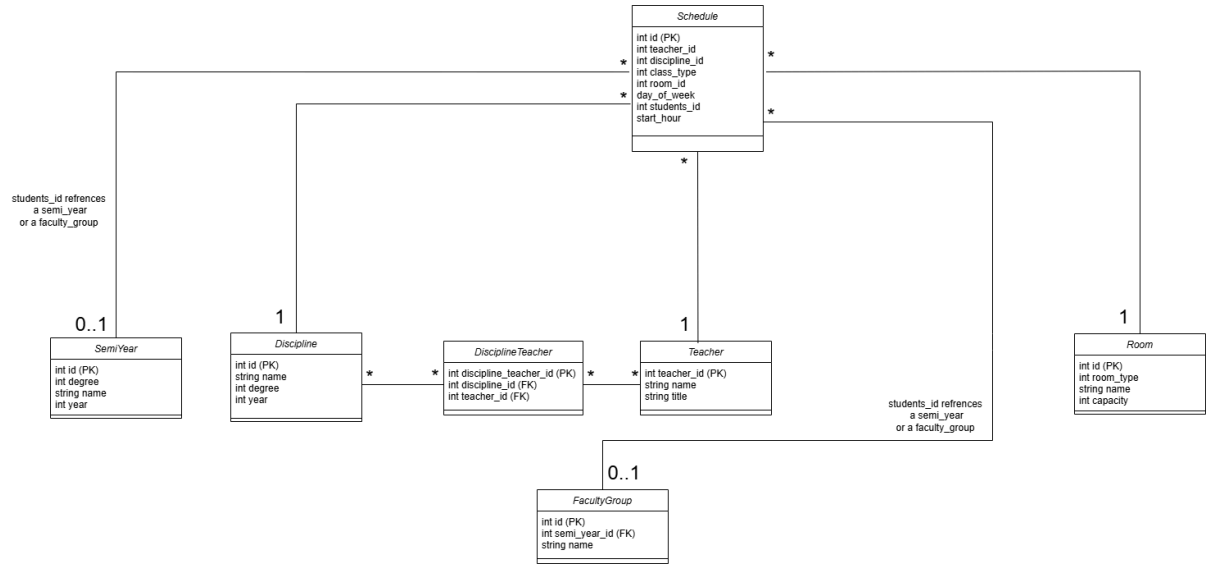
database is structured to handle data management for an academic scheduling system, covering entities such as rooms, teachers, disciplines, and schedules. The schema follows a relational design with foreign key relationships and indexed optimization for query performance.

1. **Entities and Relationships**:
    - The database contains several core entities related to academic scheduling:
        - **Teachers** and **Disciplines**: Linked via a many-to-many relationship table (`TeacherDiscipline`).
        - **Faculty Groups**: Associated with a **SemiYear**, representing sub-divisions of students.
        - **Rooms**: Define physical locations with details such as capacity and type (e.g., laboratory, course).
        - **Schedule**: Represents planned classes and facilities allocation, connecting teachers, disciplines, rooms, and students.
2. **Primary Tables**:
    - **Room**:
        - Columns: `id`, `name`, `capacity`, `room_type`
        - Stores information about physical rooms in the facility.
    - **SemiYear**:
        - Columns: `id`, `name`, `degree`, `year`.
        - Represents specific periods of study (e.g., Bachelor's first year).
    - **FacultyGroup**:
        - Columns: `id`, `name`, `semi_year_id`
        - Organizes students for scheduling purposes, linked to `SemiYear`.
    - **Discipline**:
        - Columns: `id`, `name`, `degree`, `year`.
        - Represents courses or academic disciplines.
    - **Teacher**:
        - Columns: `id`, `name`, `title`.
        - Stores data about instructors.
    - **TeacherDiscipline**:
        - Columns: `id`, `teacher_id`, `discipline_id`.
        - A mapping table to establish which teachers are associated with specific disciplines.
    - **Schedule**:
        - Columns: `id`, `teacher_id`, `discipline_id` , `students_id`, `,class_type,room_id,day_of_week,start_hour`

        - Core table which represents the scheduled classes, integrating teachers, disciplines, rooms, and time slots.
3. **Indexes**:
    - Indexes are added to key columns for faster lookup and efficient querying:
        - on `Schedule` for teacher-discipline queries. `idx_schedule_teacher_discipline`
        - on `Schedule` for room-based queries. `idx_schedule_room`
        - on `FacultyGroup` for quick lookups. `idx_group_faculty_group`

Each of the database tables has a corresponding entity, represented as a Java class in our application.

Database schema:



- **Persistence Managers** handle CRUD operations specific to their respective domain models. These managers interact with **Repositories**, which encapsulate database interaction logic.
- **CSV Data Import**:
    - The provides utility methods for importing data into the database from predefined `.csv` resources.
    - Data is preloaded into tables from CSV files for testing/demo purposes (e.g., initial lists of teachers, disciplines, etc.).

4. Technologies Used:

- **Java SE 17** for the core application logic and structure.
- **Javalin** framework for handling RESTful API requests.
- **SQLite** for data persistence.
- **CSV data importing** for initial data population.
- **SvelteKit 5** and **Tailwind** for the UI Application

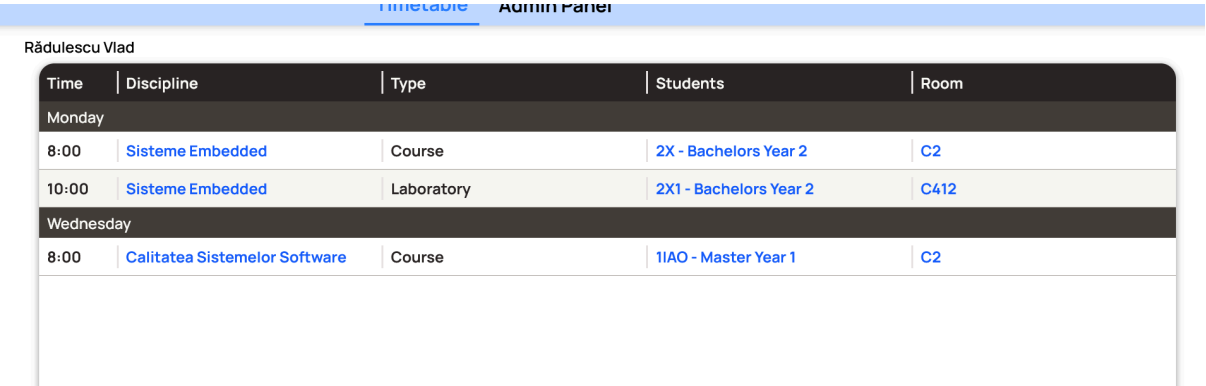
5. Application frontend and UI

The application's frontend can be started using the file 'Start_UI.bat' that will install any *npm* dependencies and start the UI server. The main database server must be open for this app to work properly. Design-wise, it consists in 2 main pages:

- Timetable

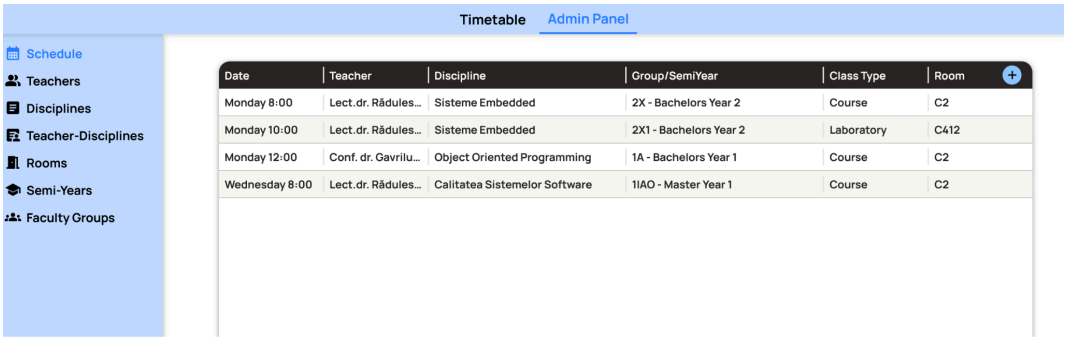


In this page, the current timetable schedule can be viewed, filtering the results by teachers, disciplines, rooms, semi-years and groups.



The timetable is sorted by date and groups every weekday, omitting the ones that don't have any class scheduled. All table data that references another entity can be clicked, redirecting the user to its corresponding timetable page, making navigation easier.

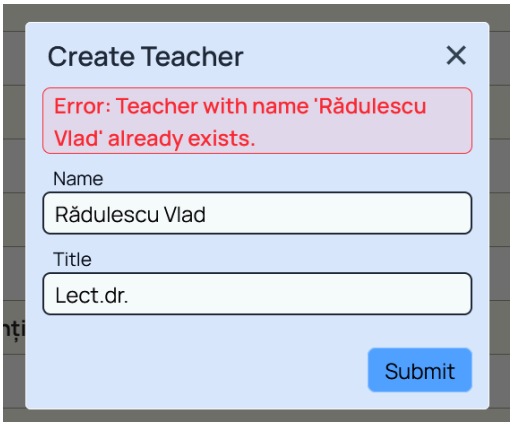
- Admin Panel



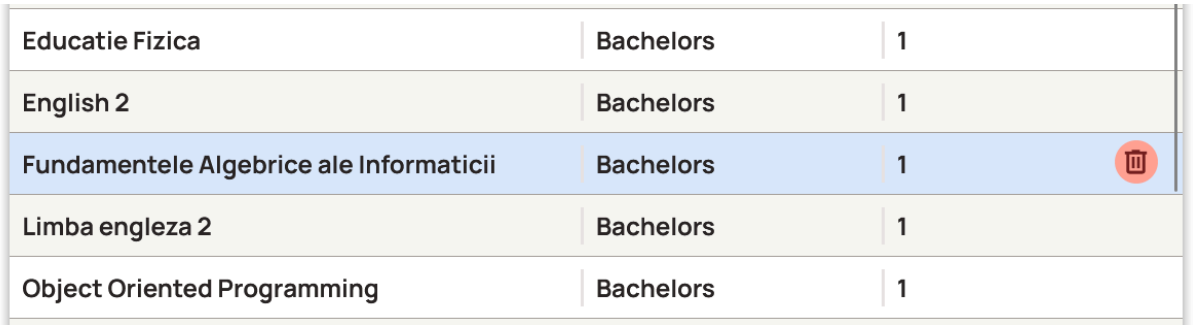
In the admin panel, the user can configure the current timetable schedule, and its entities. The entries are sorted by date or alphabetically, whether it is a schedule or an entity.

Clicking the '+' button will bring up a menu to create a new entry. The same menu is used for editing, by clicking the respective entry in the table.

Any incorrect input will alert the user with a corresponding error report. Every successful operation will instantly refresh the table with its updated data.



Hovering over a table entry will reveal a 'delete' button to remove it from the database. Any unsuccessful operation will deny the request and alert the user with a detailed error that appears in the form of a popup.



Phase 2 - unit testing

1. Purpose of Unit Testing

The purpose of unit testing in this project is to verify the correctness and robustness of the application logic in isolation from the underlying infrastructure (e.g., database). Unit tests were written to detect bugs early and ensure that each component behaves as expected under different scenarios. The goal was to ensure correctness, robustness, and isolation of the persistence layer, data import logic, and API endpoints.

2. Testing Strategy

The project uses two complementary approaches to unit testing:

- **Mock-based testing**: Dependencies such as repositories are mocked using libraries (e.g., Mockito). This allows fine-grained control over test inputs and behavior verification.
- **Fake repositories**: Instead of mocking, lightweight in-memory fake implementations of repositories were created. This approach allows for more realistic simulations of repository behavior without requiring a real database.

The following components were covered:

- **Entities**: Ensured correct behavior of domain classes and validation logic.
- **Managers**: Validated business rules, such as uniqueness constraints, cascading deletions, and consistency rules.
- **Database Integration**: Used fake repositories to simulate database behavior in-memory.
- **CSV Data Importer**: Validated correctness and robustness of CSV file parsing and entity creation.
- **API Controllers**: Verified endpoint behavior using unit-level web tests.
- **Query Logic**: Ensured dynamic filters and custom queries work as intended.

3. Tools and Frameworks Used

- **JUnit 5**: for writing and executing tests.
- **Mockito**: for creating mocks and verifying interactions.
- **JUnit Jupiter Assertions**: for expressive assertions (optional).
- Custom *fake repository* implementations used as stand-ins for real repositories.

4. Validated Rules per Component

1. `DisciplineManager`

- Prevented duplicates by name.
- Cascade delete of `TeacherDiscipline` when a discipline is deleted.

2. `FacultyGroupManager`

- Allowed prefixes: `A`, `B`, `E`, `X`.
- Prevented duplicate groups (by name + year).
- Cascade delete of `Schedule` entries on group deletion.
- Name validation: one uppercase letter + digit (e.g., A1, B2).
- Bachelor = max 3 years, Master = 2 years, PhD = any positive number.
- Ensured group-studyYear match in education level (e.g., master group only with master year).

3. `RoomManager`

- Name must be non-empty and unique.
- Capacity must be positive.
- Cascade delete of schedules when room is removed.

4. `ScheduleManager`

- Days allowed: Monday to Friday.
- Time allowed: between 08:00 and 20:00.
- `course`, `seminar`, `lab` must last exactly 2 hours (exams are exempt).
- Room type must match activity type:
  - Course → Room type: course
  - Lab → Room type: lab
  - Seminar → any room
- No overlapping schedules for:
  - Same room
  - Same teacher
  - Same group (including groups of same year)
- Coursing logic:
  - All groups from the same year must be free during the course time.
  - Each discipline must have one `course` per study year.
  - Labs/seminars should match the number of groups in the discipline's year.
- Prevented:
  - Scheduling a discipline for a different year/group level.
  - Assigning a Master-level discipline to a Bachelor group (and vice versa) or PhD discipline.

- Scheduling the same `course` more than once per week.

5. `StudyYearManager`

- Bachelor: specialization = "Computer Science", maxYear = 3
- Master: specialization in {"ISS", "SD", "IAO", "SI", "Advanced Studies"}, maxYear = 2
- Doctorate: any specialization, positive maxYear
- Cascade delete groups in the year when year is deleted
- Cannot update the specialization of an existing StudyYear
- Prevent duplicate StudyYear entries (same level + year)

6. `TeacherDisciplineManager`

- Both teacher and discipline must be non-null.
- Prevented assigning the same discipline to the same teacher multiple times.

7. `TeacherManager`

- Names must be unique.
- Cascade delete all `TeacherDiscipline` entries on teacher removal.

8. Global Validations

- Each year must have one `course` per discipline.
- Each discipline should appear only once per week.
- Courses cannot be scheduled multiple times per week.

9. Frontend Support (UX enforced logic)

- Group creation includes selecting education level from a dropdown.
- The drop down filters `StudyYear` options based on selected level.

5. Sample Unit Test

1. Using fakes

TeacherDiscipline: this test checks a key integrity feature: if a discipline is associated with a teacher, it should not be deletable, and the application should throw a relevant exception.

```
@Test
void testAddTeacherDisciplineAndRemoveDisciplineFails() {

   // 1. Add a teacher and a discipline
   disciplineManager.addDiscipline("CSS", Degree.Master, 1);
   teacherManager.addTeacher("Name", "title");
```

```
    var discipline = disciplineManager.getAll().get(0);
    var teacher = teacherManager.getAll().get(0);


    // 2. Create an association between the teacher and the discipline
    teacherDisciplineManager.addTeacherDiscipline(teacher.getId(),
discipline.getId());


    // 3. Check that the association exists
    List<TeacherDiscipline> all = teacherDisciplineManager.getAll();
    assertEquals(1, all.size());
    assertEquals(teacher.getId(), all.get(0).getTeacherId());


    // 4. Attempt to delete the discipline (should fail)
    RuntimeException ex = assertThrows(RuntimeException.class, () ->
disciplineManager.remove(discipline.getId()) );


    // 5. Confirm the error message
    assertTrue(ex.getMessage().contains("Discipline is still associated with
a teacher."));
}
```

1. What This Test Verifies:
    - The integration between `TeacherManager`, `DisciplineManager`, and `TeacherDisciplineManager`;
    - That a discipline cannot be deleted while it has an active association;
    - That error message is informative and accurate.
2. Why This Is Relevant:
    - Demonstrates an important business rule: referential integrity;
    - Uses a fake repository setup (no real database required);
    - Clearly shows how the test is configured and how behavior is validated.

2. Using mocks

ScheduleManager: This test ensures that all business rules and validation constraints are respected when adding a schedule. It uses mocks for all dependencies, simulating database and manager behavior without needing a real database.

Initial Test Context: A predefined set of entities is configured before the test to simulate a realistic scheduling environment. They are injected into their respective manager or repository mocks using `when(...).thenReturn(...)`. They serve as the foundation for validating correct schedule creation and detecting rule

violations such as overlapping classes, mismatched room types, or invalid associations.

```
@Test
public void testAdd() {
   try(var mockedStatic = mockStatic(Database.class)) {
       mockedStatic.when(Database::getInstance).thenReturn(mockDatabase);

       when(mockRepository.newEntity()).thenReturn(
           new Schedule("ID", "", "", 0, "", "", 0, 0)
       );


       // 1. Successfully add a valid schedule
       assertDoesNotThrow(() -> manager.addSchedule("t2", "d2",
ClassType.Laboratory, "r2", "f1", DayOfWeek.Monday, 18));

       // 2. Validate input fields (empty values)
       assertThrows(AssertionError.class, () -> manager.addSchedule("",
"d1", ClassType.Laboratory, "r2", "f1", DayOfWeek.Monday, 18));
       assertThrows(AssertionError.class, () -> manager.addSchedule("t1",
"", ClassType.Laboratory, "r2", "f1", DayOfWeek.Monday, 18));
       assertThrows(AssertionError.class, () -> manager.addSchedule("t1",
"d1", ClassType.Laboratory, "", "f1", DayOfWeek.Monday, 18));
       assertThrows(AssertionError.class, () -> manager.addSchedule("t1",
"d1", ClassType.Laboratory, "r2", "", DayOfWeek.Monday, 18));

       // 3. Validate class hour bounds
       assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Course, "r1", "s1",
DayOfWeek.Monday, 6));
       assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Course, "r1", "s1",
DayOfWeek.Monday, 22));
       assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Course, "r1", "s1",
DayOfWeek.Monday, 11));




       // 4. Validate ClassType matches RoomType
       assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Laboratory, "r1", "f1",
DayOfWeek.Monday, 8));
       assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Course, "r2", "s1",
DayOfWeek.Monday, 8));

       // 5. Validate Discipline-Group compatibility
```

```
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d2", ClassType.Laboratory, "r2", "f1",
DayOfWeek.Monday, 18));
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d3", ClassType.Laboratory, "r2", "f1",
DayOfWeek.Monday, 18));

        // 6. Validate Discipline-Teacher association
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t2", "d1", ClassType.Laboratory, "r2", "f1",
DayOfWeek.Monday, 18));

        // 7. Validate week-related group restrictions
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Course, "r1", "s2",
DayOfWeek.Monday, 18));
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Laboratory, "r2", "f2",
DayOfWeek.Monday, 10));

        // 8. Detect scheduling overlaps
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Laboratory, "r2", "f3",
DayOfWeek.Monday, 8));
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t2", "d2", ClassType.Laboratory, "r2", "f1",
DayOfWeek.Monday, 10));
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t3", "d4", ClassType.Seminary, "r3", "f2",
DayOfWeek.Monday, 10));
        assertThrows(DatabaseException.class, () ->
manager.addSchedule("t3", "d4", ClassType.Seminary, "r3", "f2",
DayOfWeek.Monday, 8));

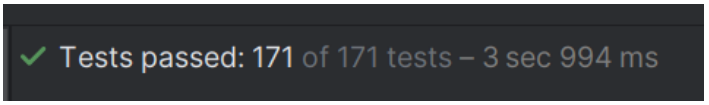

        //assertThrows(DatabaseException.class, () ->
manager.addSchedule("t1", "d1", ClassType.Laboratory, "r2", "f1",
DayOfWeek.Monday, 18));
        verify(mockRepository, times(1)).persist(any(Schedule.class));
    }
}
```

6. Test Coverage Overview

The project demonstrates **high test coverage**, ensuring robust verification of both core logic and edge cases. Unit tests verify individual method correctness (validation, cloning, scheduling logic), while integration tests ensure proper interaction with the database layer. Special attention has been given to error handling, schedule conflicts, and class-type constraints, confirming the system behaves predictably under all expected scenarios.

1. **Overall**

Element	Class, %	Method, %	Line, %	Branch, %
fii.css	95% (41/43)	94% (282/2...	90% (1076/...	69% (387/5...
api	90% (9/10)	88% (56/63)	90% (205/2...	95% (21/22)
database	100% (32/32)	96% (226/2...	91% (871/9...	68% (366/5...
Main	0% (0/1)	0% (0/1)	0% (0/4)	100% (0/0)



2. **Persistence**
    a. **entities**

Element	Class, %	Method, %	Line, %	Branch, %
fii.css.database.persistence.entities	100% (11/11)	84% (91/108)	74% (160/214)	58% (14/24)
ClassType	100% (1/1)	100% (5/5)	100% (10/10)	100% (0/0)
DayOfWeek	100% (1/1)	100% (5/5)	100% (12/12)	100% (0/0)
Degree	100% (1/1)	80% (4/5)	66% (6/9)	100% (0/0)
Discipline	100% (1/1)	75% (9/12)	68% (15/22)	50% (2/4)
FacultyGroup	100% (1/1)	87% (7/8)	73% (11/15)	100% (0/0)
Room	100% (1/1)	83% (10/12)	72% (16/22)	50% (2/4)
RoomType	100% (1/1)	100% (5/5)	100% (10/10)	100% (0/0)
Schedule	100% (1/1)	88% (22/25)	76% (42/55)	66% (8/12)
SemiYear	100% (1/1)	76% (10/13)	59% (16/27)	50% (2/4)
Teacher	100% (1/1)	77% (7/9)	68% (11/16)	100% (0/0)
TeacherDiscipline	100% (1/1)	77% (7/9)	68% (11/16)	100% (0/0)

    b. **managers**

Element	Class, %	Method, %	Line, %	Branch, %
fii.css.database.persistence.managers	100% (8/8)	92% (60/65)	90% (302/33...	74% (221/29...
AbstractEntityManager	100% (1/1)	100% (2/2)	100% (3/3)	100% (0/0)
DisciplineManager	100% (1/1)	88% (8/9)	80% (40/50)	61% (27/44)
FacultyGroupManager	100% (1/1)	100% (9/9)	100% (40/40)	85% (29/34)
RoomManager	100% (1/1)	88% (8/9)	82% (34/41)	52% (18/34)
ScheduleManager	100% (1/1)	100% (9/9)	93% (76/81)	79% (67/84)
SemiYearManager	100% (1/1)	100% (10/10)	100% (51/51)	91% (42/46)
TeacherDisciplineManager	100% (1/1)	75% (6/8)	87% (28/32)	69% (18/26)
TeacherManager	100% (1/1)	88% (8/9)	83% (30/36)	66% (20/30)

    c. **repositories** (when using fake repositories)

Element	Class, %	Method, %	Line, %	Branch, %
fii.css.database.persistence.repositories	12% (1/8)	52% (9/17)	80% (104/130)	75% (53/70)
AbstractRepository	100% (1/1)	90% (9/10)	84% (104/123)	75% (53/70)
DisciplineRepository	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
FacultyGroupRepository	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
RoomRepository	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
ScheduleRepository	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
SemiYearRepository	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

d. csv data importer and database

Element	Class, %	Method, %	Line, %	Branch, %
fii.css	41% (18/43)	18% (54/297)	21% (256/11...	13% (77/555)
api	0% (0/10)	0% (0/63)	0% (0/226)	0% (0/22)
database	56% (18/32)	23% (54/233)	26% (256/9...	14% (77/533)
persistence	55% (16/29)	11% (23/194)	4% (31/694)	0% (1/396)
CsvDataImporter	100% (1/1)	100% (15/15)	91% (137/150)	55% (62/111)
Database	100% (1/1)	69% (16/23)	80% (88/110)	53% (14/26)
DatabaseException	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

3. API

a. controllers

Element	Class, %	Method, %	Line, %	Branch, %
fii.css.api.controllers	100% (8/8)	100% (49/49)	100% (187/1...	100% (14/14)
AbstractController	100% (1/1)	100% (1/1)	100% (2/2)	100% (0/0)
DisciplineController	100% (1/1)	100% (7/7)	100% (27/27)	100% (2/2)
FacultyGroupController	100% (1/1)	100% (7/7)	100% (25/25)	100% (2/2)
RoomController	100% (1/1)	100% (7/7)	100% (27/27)	100% (2/2)
ScheduleController	100% (1/1)	100% (7/7)	100% (35/35)	100% (2/2)
SemiYearController	100% (1/1)	100% (7/7)	100% (27/27)	100% (2/2)
TeacherController	100% (1/1)	100% (7/7)	100% (25/25)	100% (2/2)
TeacherDisciplineController	100% (1/1)	100% (6/6)	100% (19/19)	100% (2/2)

b. query

Element	Class, %	Method, %	Line, %	Branch, %
fii.css.api	10% (1/10)	11% (7/63)	7% (18/226)	31% (7/22)
controllers	0% (0/8)	0% (0/49)	0% (0/187)	0% (0/14)
Query	100% (1/1)	100% (7/7)	94% (18/19)	87% (7/8)
RestApi	0% (0/1)	0% (0/7)	0% (0/20)	100% (0/0)

Phase 3 - use of assertions

- Throughout the codebase, assertions are used to guarantee critical conditions, especially in areas where incorrect states could lead to inconsistent behavior or system crashes.
- Example:
    - Singleton Initialization:
        - The database's singleton (`Database.getInstance()`) uses assertions to ensure the single instance is correctly initialized.
        - Example assertion:

```
assert INSTANCE != null : "Database instance is null";
```

- Database Connection Validation:
    - Before performing operations on the database, the connection is asserted to ensure it is properly established.
    - Example assertion:

```
assert connection != null : "Database connection is null";
```

- Assertions play a pivotal role in validating the correctness of logic during automated testing.
- Libraries like JUnit are utilized, with common assertion methods like:
    - `assertEquals`: Ensure actual output matches the expected output.
    - `assertThrows`: Verify that a specific exception is thrown for invalid inputs.
    - `assertTrue` / `assertFalse`: Check boolean conditions.
    - `assertNotNull`: Validate that objects are not `null`.
- Examples in the tests:

```
disciplineManager.addDiscipline("CSS", Degree.Master, 1);

var all = disciplineManager.getAll();

assertEquals(1, all.size());

assertEquals("CSS", all.get(0).getName());
```

- Ensuring an exception is thrown for invalid data:

```
RuntimeException ex = assertThrows(RuntimeException.class, () ->
   disciplineManager.addDiscipline("CSS", Degree.Master, 5));
assertTrue(ex.getMessage().contains("Year must be less or equal"));
```

- In mock-heavy tests, such as those involving invalid or manipulated data, assertions validate the interaction with mocked objects:
    - Verification:

        - Ensure correct calls are made to dependencies, like repository methods (, `merge`, or `delete`), following specific logic paths. `persist`
        - Example:

```
verify(mockRepository, times(1)).persist(any(FacultyGroup.class));
```

- Condition Validation:
    - Verifying dynamic states or returned values of mocks using assertions adds layers of validation to the test.
- Assertions are subtly embedded in helper methods where invalid inputs could break the application:
    - Methods like `Query.integerParam` or `Query.stringParam` ensure input parameters from requests are correctly formatted and non-null.
        - Example:

```
assertThrows(DatabaseException.class, () ->
   Query.integerParam(mockContext, "age"));
```

Benefits of Using Assertions

1. **Early Bug Detection**:
    - Assertions prevent invalid states or configurations from propagating and causing larger issues down the line.
    - Critical areas like database setup and essential object initialization rely heavily on these validations.
2. **Improved Code Quality**:
    - Tests with comprehensive assertions ensure code is reliable, maintainable, and less prone to regressions.
3. **Clear Diagnostics**:
    - Well-written assertions provide meaningful error messages, aiding debugging efforts when something breaks.

Contributions

For the initial phase:

- Database: Stefana and Florin
- Backend: Stefana, Diana and Florin; we each picked tables from the database and implemented the corresponding entities and managers for them
- API: Florin
- Frontend and UI: Florin

For the second phase(testing): Diana, Stefana and Florin. We each picked classes to implement tests for

For the third phase(assertions): Florin (Diana and Stefana - review)

For the fourth phase(documentation): Diana, Stefana and Florin (we each wrote in the documentation about our implementations)